

API REST avec npm, JavaScript et TypeScript

Ce TP propose la création d'une API REST *HelloWorld* en utilisant npm, JavaScript et TypeScript.

Il permet de mettre en évidence la gestion des packages JS au travers des fichiers `package.json`, `package-lock.json` et des commandes `npm` associées.

Il démontre également les capacités de gestion d'un projet JS proposées par `npm`.

Il propose enfin une brève introduction au langage Typescript.

- 1. Prérequis
- 2. Initialiser un projet npm
- 3. Installer Typescript
- 4. Configurer Typescript
- 5. Créer le code source de l'API
- 6. Installer les dépendances
- 7. Configurer les scripts npm
- 8. Exécuter l'API
- 9. Tester l'API
- 10. Gestion de configuration
 - 10.1. Enregistrement du répertoire `node_modules/`
 - 10.2. Initialisation d'un dépôt fraîchement cloné
 - 10.3. Bonnes pratiques
- 11. Annexes
 - 11.1. Documentation utile

1. Prérequis

- Node.js installé (version 18 ou supérieure)

Installation de Node.js sous Linux

```
sudo apt install node npm
node --version
npm --version
```

- `curl` installé (en option, pour tester en ligne de commande)
- Un terminal (Linux/MacOS) ou PowerShell/Command Prompt (Windows)

GitBash pour Windows

Sur Windows, une option rapide et pratique est d'utiliser GitBash.

- Un éditeur de code (VS Code, ...)

2. Initialiser un projet npm

Ouvrir un terminal et créer un nouveau dossier pour le projet :

```
mkdir "TP - REST API npm-ts"  
cd "TP - REST API npm-ts"
```

Initialiser un projet npm dans le répertoire :

```
npm init
```

Renseigner les différents champs demandés.

Observer le fichier `package.json` généré.

3. Installer Typescript

Installer TypeScript et les types de base pour Node.js :

```
npm install typescript @types/node --save-dev
```

Option `--save-dev`

L'option `--save-dev` permet de préciser que la dépendance requise concerne l'environnement de développement uniquement, et non le logiciel produit.

Lorsque la dépendance est nécessaire pour l'exécution (comme pour l'installation d'`express` plus tard), ne pas utiliser l'option `--save-dev`.

Observer les modifications dans le répertoire projet :

- Fichier `package.json` :

Une section `devDependencies` a été ajoutée, indiquant les dépendances attendues : `typescript` et `@types/node`.

Spécification des versions des dépendances

Noter la forme utilisée pour la spécification des versions attendues des dépendances :

```
"devDependencies": {
  "@types/node": "^25.0.10",
  "typescript": "^5.9.3"
}
```

Le caractère `^` précédant le numéro de version signifie *à partir de*.

Voir <https://docs.npmjs.com/about-semantic-versioning> pour plus de détails sur la manière de spécifier les versions requises dans les fichiers `package.json`.

- Un répertoire `node_modules/` est apparu, avec une série de sous-répertoires, dont `node_modules/typescript/` et `node_modules/@types/`.

Noter l'existence d'autres sous-répertoires dans `node_modules/`.

Contenu des répertoires `node_modules/`

C'est dans les répertoires `node_modules/` que `npm` installe les packages téléchargés.

C'est également à partir de ces répertoires que `node` recherche les dépendances de packages nécessaires à l'exécution.

- Un fichier `package-lock.json` est apparu.

Observer son contenu.

On retrouve des informations pour les packages `typescript` et `@types/node` requis, mais également pour les packages dont ils dépendent eux-mêmes, d'où les téléchargements supplémentaires qu'on a pu observer dans le répertoire `node_modules/`.

Contenu des fichiers `package-lock.json`

Les fichiers `package-lock.json` consignent le résultat effectif de la résolution des dépendances réalisée par `npm`.

Les numéros de version attachés à chaque package ne contiennent plus de caractère `^`.

Installation des outils en local projet

Noter que l'installation des outils requis pour le projet s'installent en local dans le répertoire du projet.

Cela permet notamment d'éviter des conflits de versions d'outils lorsqu'on travaille sur plusieurs projets différents sur le même poste de développement.

Installer également `ts-node` et `nodemon` pour faciliter le développement :

```
npm install ts-node nodemon --save-dev
```

Visualiser le contenu du répertoire `node_modules/`, ainsi que du fichier `package-lock.json` : on commence à avoir plus de choses...

4. Configurer Typescript

Générer un fichier de configuration TypeScript :

```
npx tsc --init
```

Observer le contenu du fichier `tsconfig.json` généré.

S'assurer des configurations suivantes dans ce fichier :

```
{
  "compilerOptions": {
    "target": "ES6",
    "module": "ESNext",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "verbatimModuleSyntax": false
  }
}
```

i Options de configuration Typescript

Cf. <https://www.typescriptlang.org/tsconfig/> (<https://aka.ms/tsconfig> dans le fichier généré) pour des explications détaillées sur les valeurs de configuration Typescript.

Spécifier également la configuration `"type": "module"` dans le fichier `package.json` pour déclarer notre projet comme un projet ESM (et non CommonJS) :

```
{
  "name": "practice-rest-api-npm-ts",
  "version": "1.0.0",
  "description": "API REST avec npm, JavaScript et TypeScript.",
  "main": "index.js",
  "type": "module", // <-- Ligne à rajouter.
}
```

i ESM v/s CommonJS

CommonJS correspond à une norme JS historique pour la gestion des modules.

Depuis ES6 (ES2015), > ESM a été introduit et devient la norme dans le monde JS.

Cf. https://www.w3schools.com/nodejs/nodejs_modules_esm.asp pour plus de détails.

5. Créer le code source de l'API

```
// src/index.ts

import express, { Request, Response } from 'express';

const app = express();
const PORT = 3000;

// Endpoint "Hello World".
app.get('/hello', (req: Request, res: Response) => {
  console.log('/hello requested');
  res.send('Hello World!');
});

// Start the server.
app.listen(PORT, () => {
  console.log(`Server started on http://localhost:${PORT}`);
});
```

Typage Typescript

Noter dans ce fichier source le typage des paramètres `req` et `res` introduit par le caractère `:`.

6. Installer les dépendances

Installer `express` en tant que dépendance d'exécution :

```
npm install express
```

Visualiser le contenu du fichier `package.json` : on voit l'apparition d'une nouvelle section `dependencies` pour les dépendances d'exécution.

Installer également les types `express` pour le développement avec Typescript :

```
npm install @types/express --save-dev
```

Visualiser le contenu du répertoire `node_modules/`, ainsi que du fichier `package-lock.json` : il commence à y avoir vraiment du monde là-dedans...

7. Configurer les scripts npm

Modifiez le fichier `package.json` pour ajouter les scripts `build`, `start` et `dev` dans la section `scripts` déjà existante :

```
{
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js",
    "dev": "nodemon --exec 'node --loader ts-node/esm src/index.ts'"
  }
}
```

npm run

La commande `npm run xxx` permet d'exécuter le script "xxx" configuré dans le fichier `package.json`.

i **nodemon & ESM**

Pour un projet en CommonJS, il suffit normalement de configurer le script `dev` avec la ligne `nodemon src/index.ts`.

Comme notre projet est un projet ESM, il faut le spécifier à `nodemon`.

Pour ce faire, on utilise l'option `--exec` qui nous permet de lancer `node` avec une option `--loader ts-node/esm`.

Des complications malheureusement assez fréquentes dans l'écosystème JS.

8. Exécuter l'API

Pour démarrer l'API en mode développement :

```
npm run dev
```

Mode développement et rechargement automatique

L'intérêt du mode développement est le rechargement automatique sur modification des sources.

Modifier la ligne de log sur exécution du endpoint `/hello`, et enregistrer le fichier `src/index.ts`.

Observer les traces dans la console.

TODO : Ne fonctionne pas chez moi.

Pour compiler et démarrer l'API en mode production :

1. `npm run build`

Observer le contenu du répertoire `dist/` automatiquement créé.

Observer le contenu du fichier `dist/index.js` généré : On ne voit plus les caractères `:` qui servaient à spécifier le typage en Typescript.

Transpilation

En technos JS / Typescript, on parle plus de transpilation que de compilation, car la sortie de l'opération de build reste du langage JS interprété, et non un *byte code* machine.

La transpilation permet notamment de s'assurer d'une sortie compatible ES6, pour une meilleure compatibilité en exécution, alors qu'on développerait avec un standard JS (ECMA) supérieur.

2. `npm start`

npm start

`npm start` est un raccourci pour `npm run start`.

Note : Le raccourci `npm build` ne fonctionne pas!

Voir :

```
npm help start  
npm help build
```

9. Tester l'API

Ouvrir l'adresse "`http://localhost:3000/hello`" à partir d'un navigateur.

Ou bien en ligne de commande :

```
curl -X GET http://localhost:3000/hello
```

Le résultat obtenu devrait être :

```
Hello World!
```

10. Gestion de configuration

10.1. Enregistrement du répertoire `node_modules/`

Comme on l'a vu précédemment, le contenu du répertoire `node_modules/` est très fourni. Il n'est donc pas judicieux d'enregistrer son contenu, sous git notamment.

Répertoires `node_modules/` et `.gitignore`

Les répertoires `node_modules/` sont classiquement ignorés pour git, au moyen des fichiers `.gitignore` dédiés.

Par conséquent, quand on clone un dépôt git, le répertoire `node_modules/` est classiquement absent, et doit être reconstitué.

10.2. Initialisation d'un dépôt fraîchement cloné

Supprimer les répertoires `node_modules/` et `dist/` pour simuler le cas d'un dépôt fraîchement cloné.

Exécuter la commande `npm start`.

! Erreur d'exécution

Le programme échoue avec une erreur "Cannot find package 'express'".

Exécuter la commande `npm clean-install`.

`npm ci`

`npm ci` est un raccourci courant pour `npm clean-install`.

Voir :

```
npm help clean-install  
npm help ci
```

Exécuter la commande `npm start` à nouveau.

Succès

Le serveur se lance avec succès et se met en écoute sur l'adresse `http://localhost:3000`.

10.3. Bonnes pratiques

Pour terminer avec cet exemple, on attire l'attention sur les différences entre `npm install` et `npm clean-install`

⚠ Attention : `npm install`

Le nom de la commande est un faux-ami !

La commande `npm install` (alias `npm i`) télécharge les dépendances comme `npm clean-install` (alias `npm ci`), mais met potentiellement à jour les versions téléchargées, notamment si de nouvelles versions ont été publiées dans les dépôts de packages.

On peut considérer souhaitable de se mettre à niveau régulièrement pour des raisons de sécurité.

Toutefois, il reste primordial de maîtriser les versions des dépendances utilisées pour une version donnée de notre logiciel.

C'est pourquoi on prendra soin de suivre les recommandations suivantes :

🔒 Utilisation du fichier `package-lock.json`

1. Enregistrer le fichier `package-lock.json` sous git pour le suivre en version.
2. Utiliser la commande `npm clean-install` pour télécharger les dépendances en cohérence avec le fichier `package-lock.json` enregistré.
3. N'utiliser la commande `npm install` que pour faire une mise à niveau volontaire des dépendances, et enregistrer le fichier `package-lock.json` mis à jour en conséquence.

On notera par ailleurs que ce sont les fichiers `package-lock.json`, qui donnent les versions effectivement résolues, et non les fichiers `package.json` qui donnent des intentions de versions, qu'il faut considérer quand on fait des analyses d'impact.

11. Annexes

11.1. Documentation utile

Documentation officielle `npm` :

- <https://docs.npmjs.com/>
- <https://docs.npmjs.com/cli/>

Registry de packages `npm` :

- <https://www.npmjs.com/>